

Le Moindre Privilège (JEA vs Sudo)

Objectifs :

- Comprendre le paradigme du "No Standing Privileges" (NSP) exigé par l'ANSSI.
- Comparer l'approche de délégation Linux (**sudo**) avec l'approche Windows (JEA).
- Maîtriser l'architecture de *Just Enough Administration* (JEA) dans PowerShell.
- Configurer la transcription (Over-the-shoulder logging) pour le SOC.

1. La Fin des "Administrateurs Permanents"

1.1 Le problème du "Standing Privilege"

Donner les droits d'administrateur local de façon permanente à un utilisateur, même pour accomplir une seule tâche métier légitime, est une faille de sécurité majeure :

- **Surface d'attaque maximale** : Si le compte du technicien est compromis, l'attaquant hérite d'un accès total et illimité au serveur 24h/24.
- **Mouvements latéraux** : L'attaquant peut dumper la mémoire (LSASS), désactiver l'antivirus, ou modifier l'Active Directory.

1.2 Le Paradigme Zero Trust (NSP)

L'ANSSI et les frameworks modernes imposent le **No Standing Privileges (NSP)**. Un utilisateur ne doit avoir *aucun* droit d'administration par défaut. Il ne doit obtenir des privilèges que pour une tâche précise, sur une cible précise, et pour une durée limitée.

2. Linux vs Windows : Le comparatif architectural

Pour déléguer une action sans donner le mot de passe "root" ou "Administrateur", les deux mondes ont des approches différentes.

2.1 La méthode Linux (**sudo**)

Sous Linux, la délégation granulaire existe depuis des décennies via le fichier `/etc/sudoers`. On peut autoriser un utilisateur standard (ex: **jdupont**) à exécuter une et une seule commande avec les droits root.

- *Exemple* : `jdupont ALL=(root) NOPASSWD: /bin/systemctl restart apache2` L'utilisateur tape `sudo systemctl restart apache2`, le noyau Linux élève

temporairement ses privilèges juste pour cette exécution, puis les redescend.

2.2 La méthode Windows (JEA - Just Enough Administration)

Jusqu'à récemment, Windows ne savait pas élever les droits "à la commande" de manière native et sécurisée. Soit on était Administrateur, soit on ne l'était pas. Microsoft a introduit **JEA (Just Enough Administration)** via PowerShell 5.0 pour combler cette lacune.

JEA crée une "Capsule de confinement" (Endpoint) PowerShell sur le serveur cible. Lorsqu'un utilisateur standard s'y connecte, **JEA ne lui donne pas les droits administrateurs**. Au contraire, JEA crée un **Compte Virtuel (Virtual Account)** administrateur, invisible et temporaire, qui exécutera la commande *à la place* de l'utilisateur.

3. L'Architecture de JEA (Sous le capot)

La mise en place d'un Bastion JEA repose sur deux fichiers de configuration et le moteur WinRM (Windows Remote Management).

3.1 Le Fichier de Capacité de Rôle (.psrc - Role Capabilities)

Ce fichier définit **CE QUE** l'on a le droit de faire. C'est la liste blanche des commandes.

- Il liste explicitement les Cmdlets PowerShell (ex: **Restart-Service**), les fonctions et les exécutables externes autorisés.
- **Granularité extrême** : Il est possible de brider les paramètres d'une commande. Par exemple, autoriser **Restart-Service**, mais **uniquement** si le paramètre **-Name** vaut **dns**.

3.2 Le Fichier de Configuration de Session (.pssc - Session Configuration)

Ce fichier définit **QUI** a le droit de se connecter, et **COMMENT**.

- Il associe un groupe de sécurité Active Directory (ex: **Helpdesk_DNS**) à un fichier de capacité de rôle (.psrc).
- Il définit l'utilisation d'un **RunAsVirtualAccount**. L'utilisateur se connectera avec son propre compte AD standard (sans droits), mais ses commandes seront exécutées par un compte local virtuel généré à la volée, possédant les droits d'administration locaux du serveur cible.

3.3 Le transport : WinRM et le Mode de Langage Contraint (CLM)

La connexion s'effectue via le réseau en PowerShell Remoting (TCP 5985/5986). Dès que l'utilisateur est connecté dans la capsule JEA, son terminal PowerShell bascule

automatiquement en **Constrained Language Mode (CLM)** (Mode de langage contraint). Il lui est mathématiquement impossible d'exécuter du code malveillant en C#, d'invoquer des API Windows, de lire des variables système ou de sortir du périmètre défini par le `.psrc`.

4. Exploitation et Traçabilité pour le SOC

Une fois le point de terminaison JEA enregistré sur le serveur cible, l'exploitation est transparente pour le technicien.

1. **La Connexion** : Depuis son poste, le technicien ouvre PowerShell et tape :
`Enter-PSSession -ComputerName SRV-DNS -ConfigurationName "MaintenanceDNS"` (Où *MaintenanceDNS* est le nom de la capsule JEA que nous avons créée).
2. **L'Action** : Il ne voit plus son profil habituel. S'il tape `Get-Command`, il ne verra que les 2 ou 3 commandes que vous avez autorisées. Il peut redémarrer le service DNS.
3. **La Traçabilité (Transcript Logging)** : C'est le rêve de tout analyste SOC. JEA est nativement configuré pour générer des **Transcriptions**. Chaque touche de clavier tapée par le technicien dans la capsule JEA est enregistrée dans un fichier texte brut stocké dans un dossier sécurisé du serveur. En cas d'incident, l'architecte peut relire exactement les commandes (et leurs paramètres) tapées par l'utilisateur, à la seconde près.

Conclusion : JEA permet d'exécuter l'équivalent de `sudo` sous Windows, offrant un cloisonnement parfait et empêchant tout mouvement latéral de la part des administrateurs Tier 1 et Tier 2.